



COMILLAS

UNIVERSIDAD PONTIFICIA

ICAI

ICADE

CIHS

Sprint 1

Grupo:

Jorge Carnicero Príncipe,

Andrés Gil Vicente,

Jorge González Pérez

Tecnologías de Procesamiento Big Data

3º Grado en Ingeniería Matemática e Inteligencia Artificial

Índice

1. Introducción.....	3
2. Metodología.....	4
2.1. Respuesta a HU-1: Fuente de Datos y Alcance.....	4
2.1.1. Análisis Exploratorio de los Datos (EDA).....	4
2.2. Respuesta a HU-2: Diseño de la Arquitectura de Almacenamiento.....	8
2.2.1. Estrategia de Bucket Único.....	8
2.2.2. Jerarquía de Objetos (Partitioning).....	8
2.2.3. Formato de Archivo.....	8
2.3. Implementación Técnica (Lógica de complete.py).....	9
2.3.1. Smart Caching y Carga Incremental.....	9
2.3.2. Gestión Automática del Ciclo de Vida (Retention).....	9
2.3.3. Idempotencia en Infraestructura.....	10
3. Resultados.....	11
3.1. Descripción de los resultados.....	11
3.2. Pantallazos de la ejecución.....	11
3.3. Discusión de los resultados.....	14
4. Conclusión.....	15
4.1. Resumen del proceso.....	15
4.2. Principales logros.....	15

1. Introducción

En el marco del proyecto TradeData, se plantea el diseño e implementación de una arquitectura Big Data en AWS orientada al análisis de datos de trading de criptomonedas, combinando ingesta histórica (batch) y, en fases posteriores, ingesta y análisis en tiempo real.

En este Sprint 1 el objetivo es sentar la base del sistema:

- Definir y automatizar la obtención de un histórico de mercado con frecuencia diaria desde TradingView utilizando el exchange BINANCE.
- Diseñar e implementar la primera capa de almacenamiento en Amazon S3 mediante una organización eficiente de la información (bucket y jerarquía de carpetas/particiones) y un formato de fichero estándar (CSV).
- Adicionalmente, se ha decidido realizar un análisis exploratorio de los datos (EDA) para comprender mejor con qué tipos de datos estábamos trabajando, si había registros faltantes, si había algún tipo de tendencia resaltable o valor atípico en los datos descargados. Esto nos parece muy importante sobre todo en esta fase inicial del proyecto, ya que nos puede ahorrar mucho trabajo que tendríamos que realizar en siguientes fases del proyecto si no hiciéramos un EDA adecuado ahora.

Este Sprint 1 es muy importante porque las tareas que se realizan en él sientan las bases del proyecto y garantizan que trabajemos con datos fiables y accesibles para los siguientes sprints, donde entre otras tareas, se construirán procesos de procesamiento y análisis (por ejemplo con Glue) y cuadros de mando, reduciendo costes y complejidad operativa gracias a una ingesta incremental y una estructura de almacenamiento preparada para consultas selectivas. Es vital partir de una estructura de manejo y gestión de datos correctamente organizada y optimizada, para poder conseguir todo esto con éxito.

2. Metodología

A continuación, se explica detalladamente cómo se ha diseñado e implementado la solución a los objetivos planteados para el sprint, así como las principales decisiones que se han tomado y las tecnologías que se han empleado para ello.

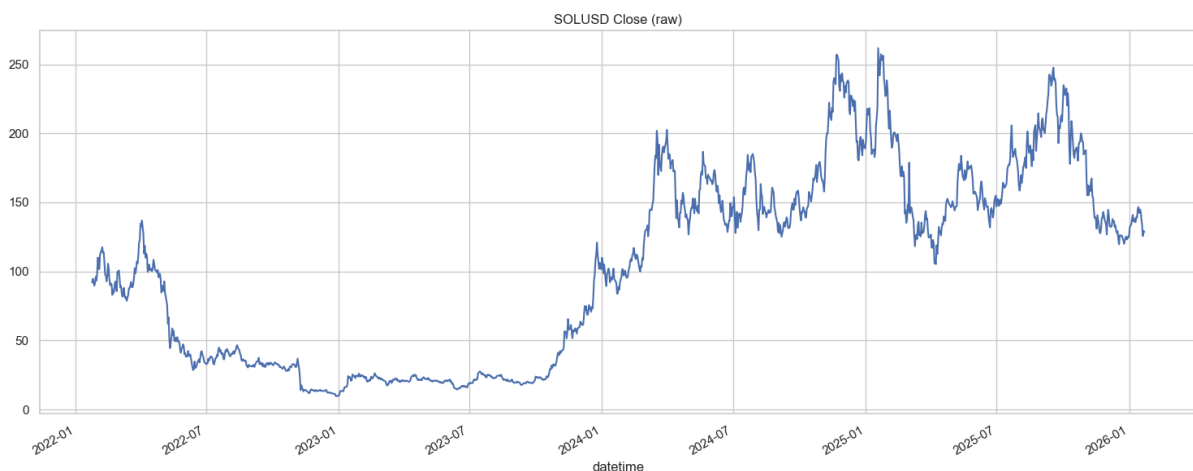
2.1. Respuesta a HU-1: Fuente de Datos y Alcance

Para cumplir con los requerimientos de negocio respecto a la obtención de datos, hemos establecido la siguiente configuración de extracción de los mismos:

- **Herramienta:** Script propio en Python integrando la librería [TradingView-Data](#).
- **Fuente de Datos:** Exchange **BINANCE**. Se ha seleccionado por ser el exchange con mayor volumen y consistencia histórica para los pares **USD**.
- **Ventana Temporal:** 4 años de histórico (retención dinámica configurada en código).
- **Frecuencia:** Datos diarios (1D).
- **Flexibilidad de acceso a Datos:** El script nos permite elegir la criptomoneda que queramos, permitiendo que el sistema sea escalable por si quisiéramos acceder a datos de más monedas, en nuestro caso únicamente queremos la de SOLANA - SOLUSD.

2.1.1. Análisis Exploratorio de los Datos (EDA)

Aunque no se ha pedido explícitamente en la historia de usuario, se ha realizado un análisis exploratorio exhaustivo de los datos cargados para validar su calidad y detectar posibles anomalías. El análisis se ha documentado en el notebook Jupyter [eda.ipynb](#) (del cual existe una copia PDF en el repositorio para una mejor visualización), que procesa el asset SOLUSD (Solana) descargado de TradingView.



Normalización y Preparación:

Los datos se normalizaron convirtiendo el índice temporal a `DatetimeIndex` y eliminando información de zona horaria para simplificar el análisis diario. Posteriormente, se consolidaron los datos por fecha aplicando agregaciones apropiadas: el precio de apertura corresponde al primer valor del día, el máximo y mínimo se calculan como el high y low del día respectivamente, el precio de cierre es el último valor del día, y el volumen se suma para obtener el total diario. El dataset resultante contiene 1,460 registros diarios que abarcan desde el 24 de enero de 2022 hasta el 22 de enero de 2026, cumpliendo con el requisito de aproximadamente 4 años de datos históricos.

Validaciones de Integridad:

La estructura del dataset confirmó la presencia de todas las columnas esperadas: symbol, open, high, low, close y volume. Todos los campos numéricos se convirtieron correctamente a tipo float64 y no se detectaron valores nulos en ninguna columna. El análisis de integridad del símbolo confirmó que todos los 1,460 registros corresponden a BINANCE:SOLUSD sin duplicados por datetime. Se validaron las reglas de coherencia OHLC verificando que el precio high sea siempre mayor o igual al máximo entre open y close, que el precio low sea menor o igual al mínimo entre open y close, que high sea mayor o igual a low, que todos los precios sean positivos, y que los volúmenes sean no negativos. Todos los registros cumplieron estas reglas sin excepciones.

Cobertura Temporal:

El análisis de cobertura temporal demostró que el dataset es completo, sin días faltantes en el rango entre la fecha mínima y máxima. Los 1,460 días esperados están presentes en su totalidad. Adicionalmente, se verificó la cobertura de fines de semana confirmando que los 416 días de sábado y domingo esperados en el período están todos presentes, validando que el mercado de criptomonedas opera de forma continua 24/7. El análisis mensual reveló cobertura completa de todos los meses en el rango, con dos meses parciales como es esperado: enero 2022 (desde el día 24) y enero 2026 (hasta el día 22). No se detectaron meses completos faltantes ni días ausentes dentro de los meses parciales.

Estadísticas y Comportamiento del Mercado:

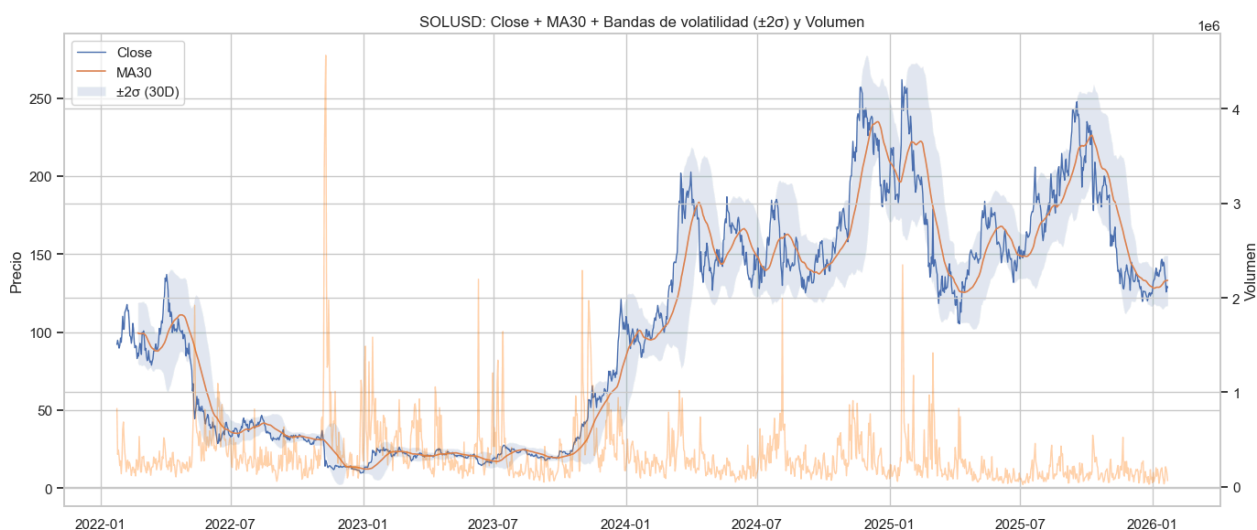
El precio de cierre de SOLUSD presenta un rango amplio desde un mínimo de \$9.64 hasta un máximo de \$261.72, con un precio medio de \$103.06 y una desviación estándar de \$69.28. Esta desviación representa aproximadamente el 67% del precio medio, evidenciando la alta volatilidad característica del mercado de criptomonedas. El volumen promedio diario es de 326,241 unidades, con picos excepcionales de hasta 4.5 millones de unidades. Los cinco precios de cierre máximos se concentran en el período de noviembre 2024 a enero 2025, con el máximo absoluto de \$261.72 alcanzado el 18 de enero de 2025, sugiriendo un rally alcista reciente. Por otro lado, los cinco días con mayor volumen de transacciones se concentran en buena medida en noviembre de 2022, en un contexto de fuertes caídas de precio asociado al colapso de FTX, lo que sugiere un aumento notable de la actividad del mercado en periodos de pánico. Aun así, también se observan picos relevantes fuera de ese mes, como un máximo puntual a finales de 2023 y otro a comienzos de 2025, indicando que los repuntes del volumen no se limitan a un único evento y pueden aparecer tanto en fases de estrés como en momentos de alta atención al mercado.

Detección de Outliers:

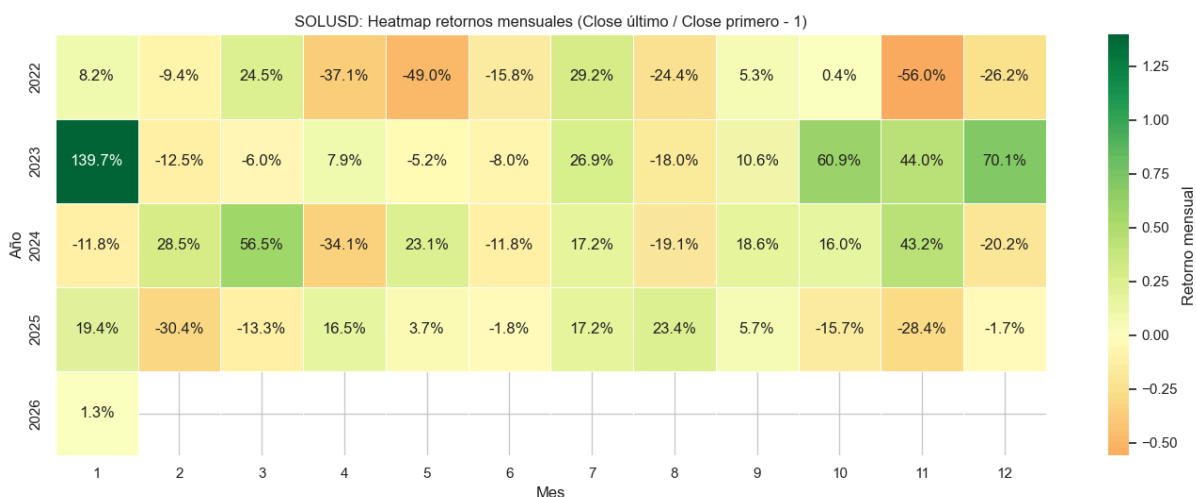
Se aplicó un análisis de outliers calculando el z-score sobre los log-returns diarios y utilizando un umbral de 6 desviaciones estándar. Se detectó un único outlier el 9 de noviembre de 2022, con un log-return de -0.551 que corresponde a una caída del precio de aproximadamente 42% en un solo día. Este evento coincide con el colapso del exchange FTX, un hecho histórico documentado que causó pánico generalizado en el mercado de criptomonedas. El outlier es legítimo y refleja un evento de mercado real, no un error de datos.

Visualizaciones Principales:

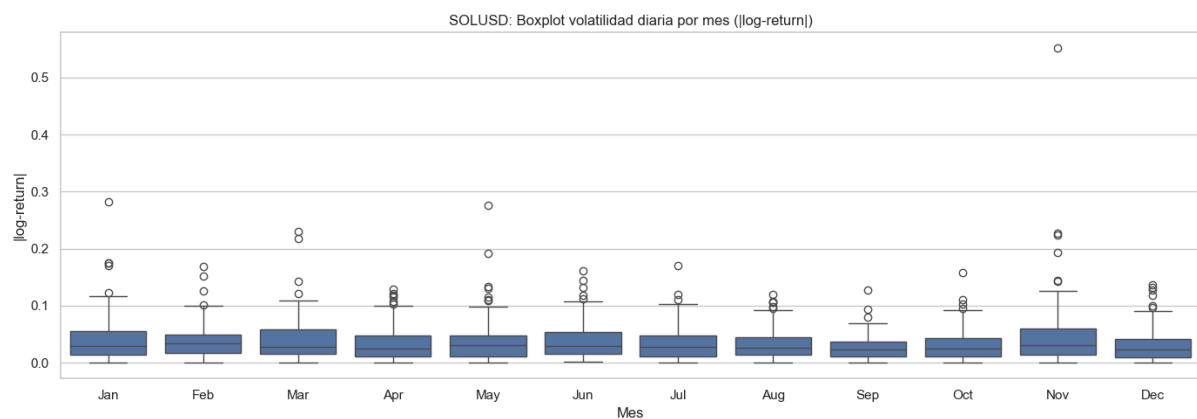
También se incluyen tres gráficos que ilustran el comportamiento temporal de SOLUSD:



La primera muestra la serie temporal del precio de cierre junto con una media móvil de 30 días y bandas de volatilidad de ± 2 desviaciones estándar, superpuesta con el volumen de trading. En esta gráfica se observan varios ciclos de volatilidad con períodos de consolidación seguidos de movimientos bruscos, donde los picos de volumen coinciden consistentemente con movimientos de precio significativos. Las bandas de volatilidad se expanden notablemente durante períodos de alta incertidumbre, particularmente en noviembre 2022 y enero 2025. La tendencia general muestra una recuperación sostenida desde los mínimos de 2022 hacia nuevos máximos históricos en el período 2024-2025.



El segundo gráfico es un heatmap de retornos mensuales que visualiza el rendimiento mes a mes. Destacan, por un lado, la subida excepcional de enero de 2023 (+139,7%), que representa el mayor retorno mensual del periodo, y por otro, la fuerte caída de noviembre de 2022 (-56,0%), asociada a un episodio de estrés del mercado (colapso de FTX). El comportamiento entre 2023 y 2024 es bastante variante, mientras que en 2025 parece más estable. En conjunto, no se aprecia un patrón estacional estable; los retornos alternan meses muy positivos y muy negativos, lo que sugiere que los movimientos responden más a eventos del mercado que a ciclos temporales predecibles.



El tercer gráfico es un boxplot que muestra la distribución de volatilidad diaria agrupada por mes. Noviembre destaca consistentemente como el mes de mayor volatilidad con medianas más altas y outliers extremos, mientras que los meses de verano tienden a mostrar menor volatilidad en promedio. La presencia de outliers en prácticamente todos los meses confirma la naturaleza inherentemente volátil del mercado crypto, y el hecho de que la volatilidad no haya disminuido con el tiempo sugiere que el asset mantiene su carácter especulativo.

Resumen de Calidad:

El dataset final contiene 1,460 muestras que cubren el período desde el 24 de enero de 2022 hasta el 22 de enero de 2026. No se encontraron días faltantes en el rango temporal, duplicados por datetime, valores nulos en columnas OHLCV, ni violaciones de las reglas de coherencia de precios. Los datos de SOLUSD son de alta calidad y están listos para ser cargados en S3 y utilizados en los siguientes sprints del proyecto para análisis de tendencias, procesamiento batch, y alimentación de dashboards de visualización.

2.2. Respuesta a HU-2: Diseño de la Arquitectura de Almacenamiento

Hemos diseñado una arquitectura **Data Lake** sobre Amazon S3, priorizando la estructura lógica de carpetas (partitioning) sobre la multiplicidad de buckets para facilitar la gestión de todo el sistema.

2.2.1. Estrategia de Bucket Único

En lugar de fragmentar la información en múltiples buckets, hemos implementado un **Bucket Centralizado**.

- **Nombre del Bucket:** `trade-data-big-daddyks-trading`
- **Región:** `eu-south-2` (España) para tener los permisos necesarios que tiene la cuenta que tenemos.
- **Justificación:** Hemos decidido tener un bucket central/principal porque queremos centralizar la entrada de datos y simplificar la gestión de permisos, permitiendo así poder asignar permisos únicamente a ese bucket. También porque esta estructura reduce la complejidad operativa, y es beneficioso para el siguiente Sprint.

2.2.2. Jerarquía de Objetos (Partitioning)

Dentro del bucket central de trading, los datos se organizan siguiendo un esquema de particionado físico compatible con **Apache Hive/Spark**. Tenemos carpetas organizadas por criptomonedas(en este caso solo una), años y meses que contienen ficheros con la siguiente forma:

- **Estructura de Clave (Key):**
`{Asset=CRIPTO}/{Year=YYYY}/{month=MM}/data.csv`
- **Ejemplo Real:** `Asset=SOLUSD/Year=2024/month=03/data.csv`
- **Justificación:**
 - **Eficiencia de Lectura (Pruning):** Al separar por Activo > Año > Mes, los futuros procesos de Spark pueden leer solo las carpetas necesarias (ej. "Solo datos de SOLUSD de 2024") sin escanear todo el bucket, lo cual es más eficiente. Además, no estamos incurriendo en sobre particionamiento, lo cual sí que podría pasar si también dividiéramos por semanas, días, horas, etc.
 - **Atomicidad:** Separar por meses evita manejar archivos CSV enormes que son difíciles de procesar o transferir.

2.2.3. Formato de Archivo

- **Tipo:** CSV (Comma Separated Values).
- **Schema:** `datetime,symbol, open, high, low, close, volume.`

2.3. Implementación Técnica (Lógica de **complete.py**)

Para la orquestación del proceso ETL, hemos desarrollado un script en Python que va más allá de una simple descarga. Hemos implementado un pipeline que desarrolla la lógica de **Ingeniería de Datos** para hacer el proceso completo y secuencial tanto de descarga como de ingesta. Además, todo esto se realiza de manera inteligente ya que hacemos uso de una memoria caché, que permite que solo tengamos que descargar los datos que nos hagan falta (teniendo en cuenta qué día es hoy y qué día se hizo la última descarga de datos) y no volvamos a descargar aquellos registros de las fechas que ya tenemos en nuestro bucket.

2.3.1. Smart Caching y Carga Incremental

Hemos implementado una lógica de "caché de estado" consultando directamente a S3 antes de descargar nada.

- **Función:** `infer_last_ingested_from_s3`
- **Lógica:** El script verifica cuál es el último archivo subido a S3 para cada activo.
 - Escenario A (Primera ejecución): No encuentra datos -> Descarga los 4 años completos, desde el día actual que lo ejecutamos.
 - Escenario B (Ejecución diaria): Encuentra que el último dato es de ayer -> Calcula el delta (día de hoy menos último día disponible en el sistema) y descarga solo el día de hoy.
- **Beneficio:** Reduce el tiempo de ejecución, que podría ser hasta de minutos si tenemos muchos años y muchas criptomonedas, pero queda reducido a segundos, logrando así reducir drásticamente el coste computacional de tener que quitar todo y volver a cargarlo, y también evita posibles bloqueos por Rate Limiting en la API de TradingView.

2.3.2. Gestión Automática del Ciclo de Vida (Retention)

El script actúa como su propio "Garbage Collector" para cumplir la política de los 4 años estrictos.

- **Función:** `retention_cleanup_months`
- **Lógica:** Antes de ingerir, el script lista las particiones existentes. Si se detectan carpetas (meses) que quedan fuera de la ventana de retención (más antiguas de 4 años), las elimina automáticamente. No obstante esto se ha configurado para poder decidir si queremos tener ese marco temporal de los cuatro años o mantener todo lo anterior y simplemente ir cargando nuevos datos. Se puede modificar con un simple booleano.
- **Beneficio:** Mantiene el Data Lake limpio y optimiza los costes de almacenamiento S3 sin intervención manual. Además es fácilmente escalable y modificable por si en el futuro queremos obtener una funcionalidad diferente.

2.3.3. Idempotencia en Infraestructura

- **Función:** `ensure_bucket_exists`
 - **Lógica:** El código verifica la existencia del bucket y, si no existe, lo crea aplicando la restricción regional (`LocationConstraint: eu-south-2`).
 - **Beneficio:** Permite desplegar el proyecto en un entorno limpio sin configuración manual previa, garantizando que la infraestructura se autogestiona (Infrastructure as Code imperativo).
-

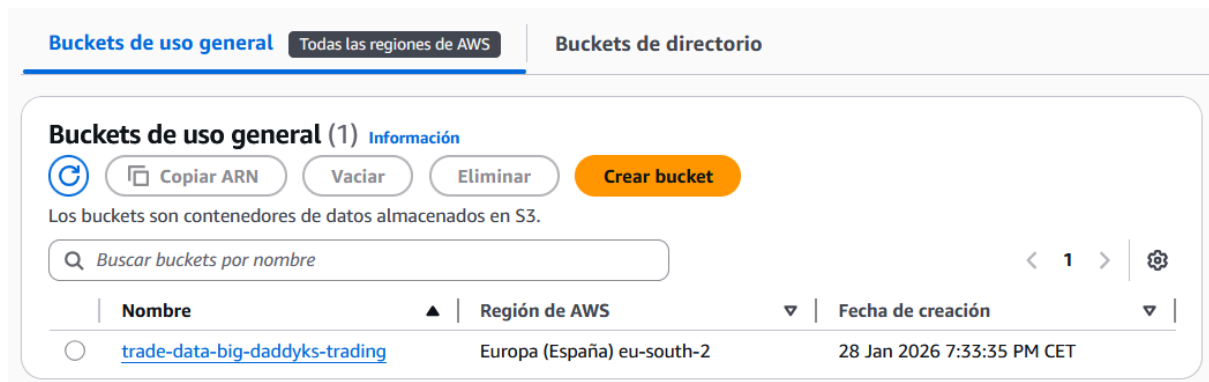
3. Resultados

3.1. Descripción de los resultados

El resultado principal obtenido es la creación automática de un bucket en Amazon S3 (trade-data-big-daddyks-trading). Como ya se ha mencionado anteriormente, dentro de este bucket, se ha generado una estructura de carpetas organizada por activo, año y mes. En el nivel más bajo de esta jerarquía encontramos los archivos CSV que contienen los datos con frecuencia diaria de la criptomoneda que estamos estudiando.

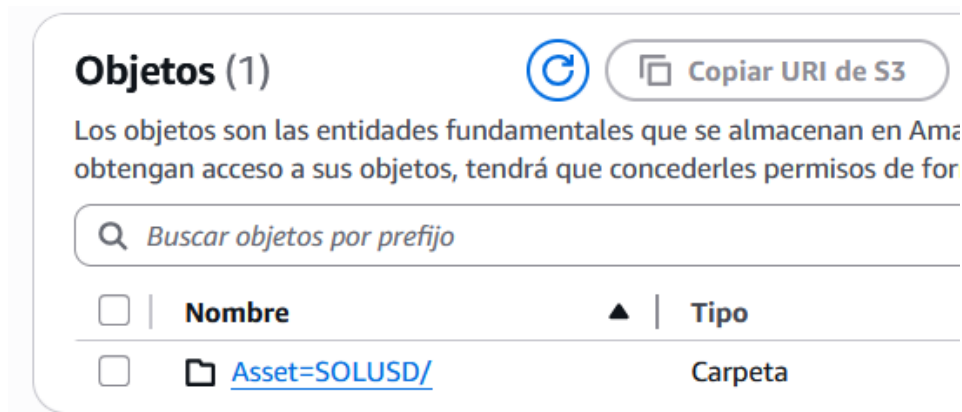
3.2. Pantallazos de la ejecución

- **Infraestructura en AWS:** Aquí mostraremos cómo queda la infraestructura desplegada en AWS y la navegación a través de las subcarpetas generadas automáticamente.



The screenshot shows the AWS S3 console interface. At the top, there are two tabs: 'Buckets de uso general' (selected) and 'Buckets de directorio'. Below the tabs, there's a section for 'Buckets de uso general (1)' with an 'Información' link. It includes buttons for 'Copiar ARN', 'Vaciar', 'Eliminar', and 'Crear bucket'. A description states: 'Los buckets son contenedores de datos almacenados en S3.' Below this is a search bar 'Buscar buckets por nombre' and a table with columns: 'Nombre', 'Región de AWS', and 'Fecha de creación'. The table contains one entry: 'trade-data-big-daddyks-trading' in the 'Europa (España) eu-south-2' region, created on '28 Jan 2026 7:33:35 PM CET'.

Nombre	Región de AWS	Fecha de creación
trade-data-big-daddyks-trading	Europa (España) eu-south-2	28 Jan 2026 7:33:35 PM CET



The screenshot shows the 'Objetos (1)' section of the AWS S3 console. It includes buttons for 'Copiar URI de S3' and a description: 'Los objetos son las entidades fundamentales que se almacenan en Amazon S3. Para obtener acceso a sus objetos, tendrá que concederles permisos de forma explícita.' Below this is a search bar 'Buscar objetos por prefijo' and a table with columns: 'Nombre' and 'Tipo'. The table contains one entry: 'Asset=SOLUSD/' with the type 'Carpeta'.

Nombre	Tipo
Asset=SOLUSD/	Carpeta

Objetos (5)

Copiar URI de S3

Los objetos son las entidades fundamentales que se almacenan en Amazon S3. Para obtener acceso a sus objetos, tendrá que concederles permisos de forma adecuada.

Buscar objetos por prefijo

☐	Nombre	Tipo
☐	year=2022/	Carpeta
☐	year=2023/	Carpeta
☐	year=2024/	Carpeta
☐	year=2025/	Carpeta
☐	year=2026/	Carpeta

Objetos (12)

Copiar URI de S3

Los objetos son las entidades fundamentales que se almacenan en Amazon S3. Para obtener acceso a sus objetos, tendrá que concederles permisos de forma adecuada.

Buscar objetos por prefijo

☐	Nombre	Tipo
☐	month=01/	Carpeta
☐	month=02/	Carpeta
☐	month=03/	Carpeta
☐	month=04/	Carpeta
☐	month=05/	Carpeta
☐	month=06/	Carpeta
☐	month=07/	Carpeta
☐	month=08/	Carpeta
☐	month=09/	Carpeta
☐	month=10/	Carpeta
☐	month=11/	Carpeta
☐	month=12/	Carpeta

En este caso aparecen 5 años cuando podríamos pensar que solo debería haber 4, porque en 2026 únicamente tenemos datos del mes de enero hasta día de hoy, y en 2022 tenemos todo. A medida que pase el tiempo, si ejecutamos el script de nuevo por ejemplo en marzo, el sistema se encargará de descargar únicamente los datos necesarios (de enero a marzo), y dependiendo de si queremos mantener la integridad de solo tener 4 años o no (controlado con el booleano comentado en apartados anteriores), eliminará los meses de enero y febrero de 2022.

- **Operación de Caché y Logs:** En esta captura de la terminal mostramos la ejecución del script. Nos mostrará por terminal la información necesaria para identificar donde se ha metido y cuando.

Salida de consola del script demostrando la lógica incremental (detectando datos previos):

Esta es la salida que obtenemos tras intentar descargar todos los datos, habiendo borrado el mes de enero de 2026. Como se puede observar, en vez de descargar los 49 meses de golpe, solo se descarga el mes faltante, tras hacer los cálculos necesarios para ubicar exactamente los datos que faltan desde la fecha de ejecución actual hasta la última fecha de ejecución.

```
C:\Users\Principal\Documents\ICAI\OneDrive - Unive
=== TradeData Batch Ingestion ===
Bucket: trade-data-big-daddyks-trading
Region: eu-south-2
Exchange: BINANCE
DRY_RUN: False
Retention cleanup: True (months < 2022-01-01)

--- Asset: SOLUSD ---
Last ingested: 2025-12-31
Need: 2026-01-01 -> 2026-01-29
Fetch from: 2026-01-01 -> 2026-01-29
Expecting value: line 1 column 1 (char 0)
Uploaded partitions: 1

Done.
```

En cambio, si borramos todos los datos (el bucket central entero) o ejecutamos el pipeline por primera vez, veremos que se nos descargan absolutamente todos los datos. De esta forma podemos ver la comparación entre los distintos comportamientos del sistema automatizado, que siempre toma la opción más eficiente, teniendo en cuenta qué datos ya hay descargados y cuáles faltan. Veamos la salida que se obtiene en la terminal si ejecutamos bajo este segundo contexto:

```
C:\Users\Principal\Documents\ICAI\OneDrive - Un
=== TradeData Batch Ingestion ===
Bucket: trade-data-big-daddyks-trading
Region: eu-south-2
Exchange: BINANCE
DRY_RUN: False
Retention cleanup: True (months < 2022-01-01)

--- Asset: SOLUSD ---
Last ingested: None
Need: 2022-01-30 -> 2026-01-29
Fetch from: 2022-01-01 -> 2026-01-29
Expecting value: line 1 column 1 (char 0)
Uploaded partitions: 49

Done.
```

3.3. Discusión de los resultados

Tras analizar las ejecuciones y ver los resultados obtenidos, podemos afirmar que el sistema se ha comportado de la manera prevista, sin registrar ningún comportamiento inesperado o errores en la lógica de ingestión.

El factor clave que ha determinado el resultado ha sido el momento de ejecución del script respecto a la última vez. Al ejecutar el proceso días después de una carga previa, el sistema detectó correctamente los datos existentes. Haciendo que en lugar de procesar y subir las 49 particiones correspondientes a todo el histórico, el script optimizó la operación subiendo únicamente 1 partición que es la faltante desde la última vez y el día actual.

Únicamente se cargarán 49 particiones cuando el sistema detecte que no existía el bucket y las subcarpetas previamente, en donde hará todo el proceso ya que el caché no detecta nada.

4. Conclusión

4.1. Resumen del proceso

El proceso se inició identificando cómo obtener los datos y gestionando la creación de buckets y carpetas con boto3, lo que nos permitió completar la fase de ingesta histórica (4 años) y cumplir las HU-1 y HU-2. Tras esta etapa inicial, dimos el salto a la automatización del entorno, integrando medidas de caché que, mediante la comparación de las fechas de los últimos datos en S3, determinan automáticamente si se requieren descargas incrementales o totales, reduciendo significativamente la carga computacional.

4.2. Principales logros

1. Ingesta datos de TradingView/Binance.
2. Gestiona la infraestructura S3 automáticamente.
3. Optimiza descargas mediante detección incremental y el uso de memoria caché.
4. Limpia datos obsoletos.
5. Permite eliminar los buckets que se deseen ejecutando el script `deletion.py` .